

CLOUD COMPUTING WITH AWS

Week 2 — Assignments & Mini Project

AWS Compute & Storage Services

Submitted by	Sneha Suresh
Program	Cloud Computing with AWS
Week	Week 2 — AWS Compute & Storage Services
Date	June 2026
Live Site	https://d30ll7dxewuby5.cloudfront.net

Week 2

Assignment 1 — Launch and Configure an EC2 Instance

Objective

Launch an Ubuntu EC2 server on AWS, connect via SSH, install a web server, and host a simple website accessible from the browser.

Steps Taken

Step 1 — Launch EC2 Instance

- AWS Console → EC2 → Launch Instance.
- Selected Ubuntu Server 24.04 LTS (Free tier eligible), Instance type: t3.micro.
- Created a new key pair (.pem) for SSH access.
- Configured Security Group — inbound: SSH (port 22) and Custom TCP (port 5000/80).
- Launched and noted the Public IPv4 address.

Step 2 — Connect via SSH using MobaXterm

Connected to the Ubuntu EC2 instance using MobaXterm with the .pem key pair:

```
ssh -i "your-key.pem" ubuntu@<EC2-PUBLIC-IP>
```

Step 3 — Install Nginx / Deploy Application

```
sudo apt update && sudo apt upgrade -y
sudo apt install nginx -y
sudo systemctl start nginx && sudo systemctl enable nginx
```

For the CampusConnect application, Docker Compose was used to run the Flask + MySQL stack:

```
git clone https://github.com/sneh20047078/CampusConnect.git
cd CampusConnect && docker compose up --build -d
```

Step 4 — Verify in Browser

Opened `http://<EC2-PUBLIC-IP>:5000` in browser — the CampusConnect application (Login, Register, Admin Dashboard) was fully accessible.

Screenshots — Assignment 1

EC2 Instance Running + Security Group

The EC2 instance CampusConnect (i-0edc19d99fc100d99) is running on t3.micro in ap-south-1a. Security Group shows inbound rules for port 5000 (HTTP) and port 22 (SSH).

8.3. EC2 Setup

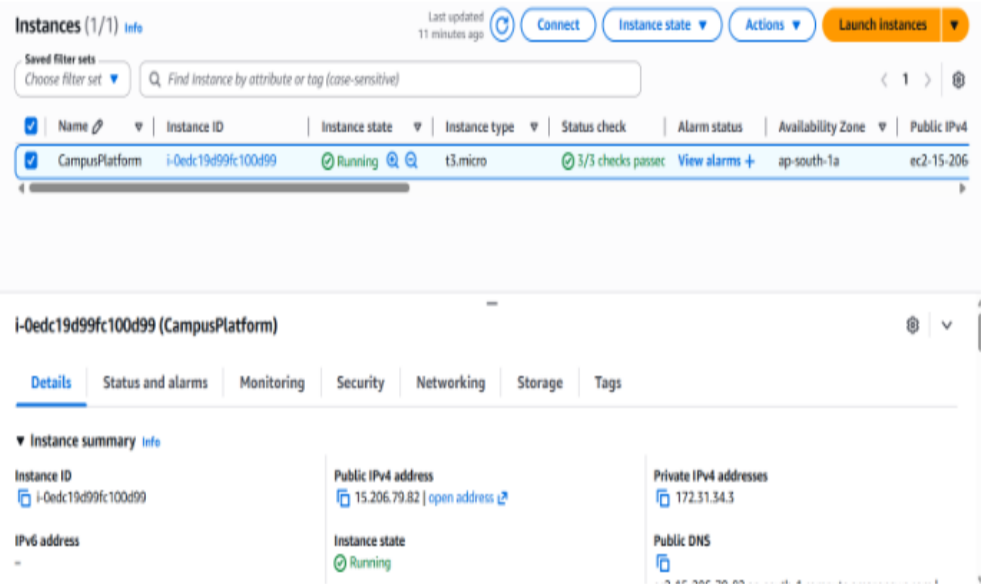


Figure 1.1: EC2 instance running — i-0edc19d99fc100d99 (CampusConnect), Public IPv4: 15.206.79.82, ap-south-1a

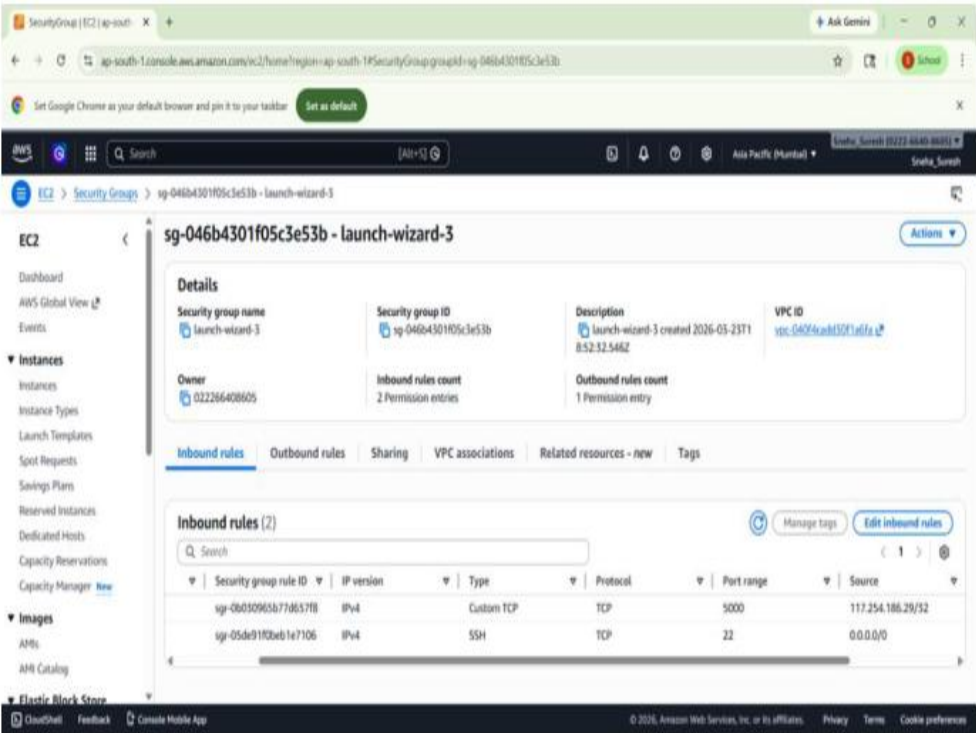


Figure 1.2: Security Group inbound rules — TCP port 5000 (app) and SSH port 22 configured

SSH Connection via MobaXterm

Successfully authenticated to the EC2 Ubuntu instance using MobaXterm. After connecting, ran apt update, cloned the GitHub repo, and started the Docker Compose stack.

using MobaXterm

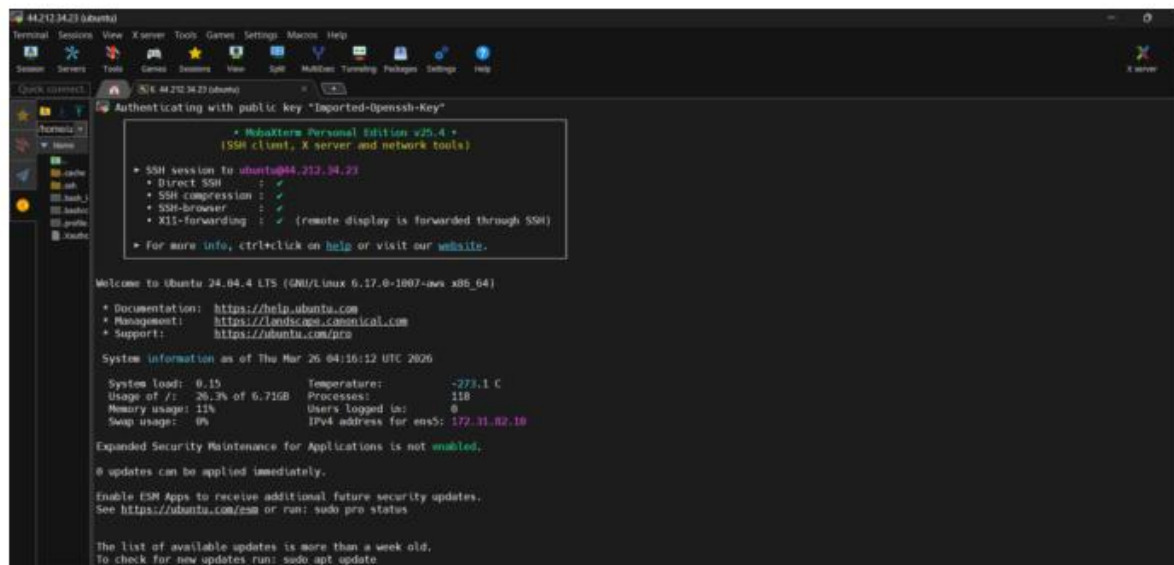


Figure 1.3: MobaXterm SSH connection to EC2 Ubuntu 24.04 LTS — IP 44.212.34.23, `sudo apt update` running

Application Running on EC2 in Browser

The CampusConnect web application is accessible at <http://15.206.79.82:5000>. The homepage, Register, and Login pages all load correctly from the EC2 public IP.

8.4 Browser Access of Application

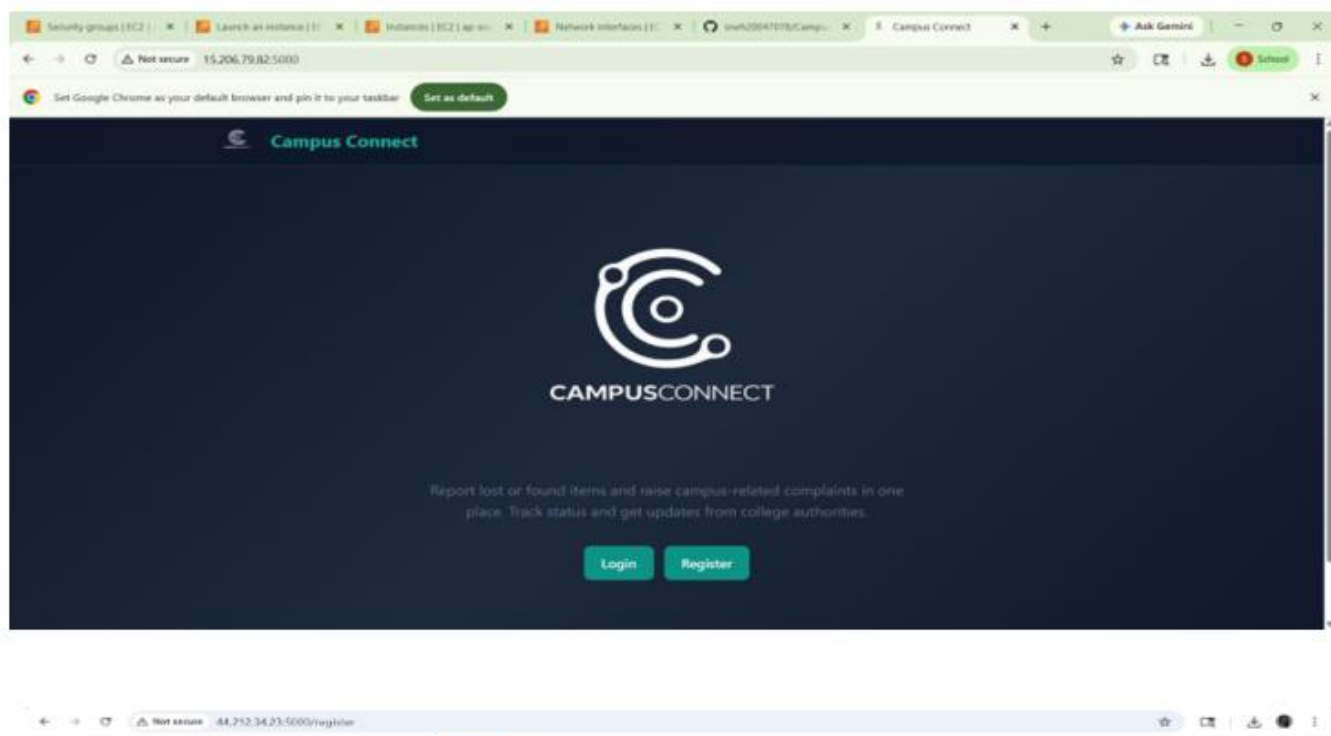


Figure 1.4: CampusConnect app running on EC2 — <http://15.206.79.82:5000> (home, register, login pages)

Key Learnings

- Provisioned and configured an EC2 Ubuntu instance on AWS with correct instance type and AMI.
- Understood Security Groups as virtual firewalls — configured inbound rules for SSH and HTTP.
- Connected to a cloud VM via SSH using .pem key pairs with MobaXterm.
- Deployed a multi-container application (Flask + MySQL) on EC2 using Docker Compose.
- Accessed a live web application served from a cloud VM via public IPv4 address.

Assignment 2 — S3 Static Website Hosting

Objective

Create an S3 bucket, upload an HTML/CSS website, configure bucket permissions for public access, and enable static website hosting.

Steps Taken

Step 1 — Create S3 Bucket

- AWS Console → S3 → Create bucket.
- Bucket name: snehasuresh-portfolio-022266408605-ap-south-1-an
- Region: Asia Pacific (Mumbai) — ap-south-1.
- Unchecked "Block all public access" to allow public read.

Step 2 — Enable Static Website Hosting

- Properties tab → Static website hosting → Enable.
- Index document: index.html.

Step 3 — Apply Bucket Policy (Public Read)

Permissions tab → Bucket Policy → pasted the following JSON to allow public s3:GetObject:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::snehasuresh-portfolio-022266408605-ap-south-1-an/*"
  }]
}
```

Step 4 — Upload Files via AWS CLI

```
aws s3 sync . s3://snehasuresh-portfolio-022266408605-ap-south-1-an --acl public-read
```

Step 5 — Verify Website

Opened the S3 website endpoint in browser to confirm portfolio loaded correctly over HTTP before setting up CloudFront.

Screenshots — Assignment 2

S3 Bucket — Objects Uploaded

The S3 bucket shows 9 objects uploaded including index.html, styles.css, and assets — all in Standard storage class, uploaded May 29, 2026.

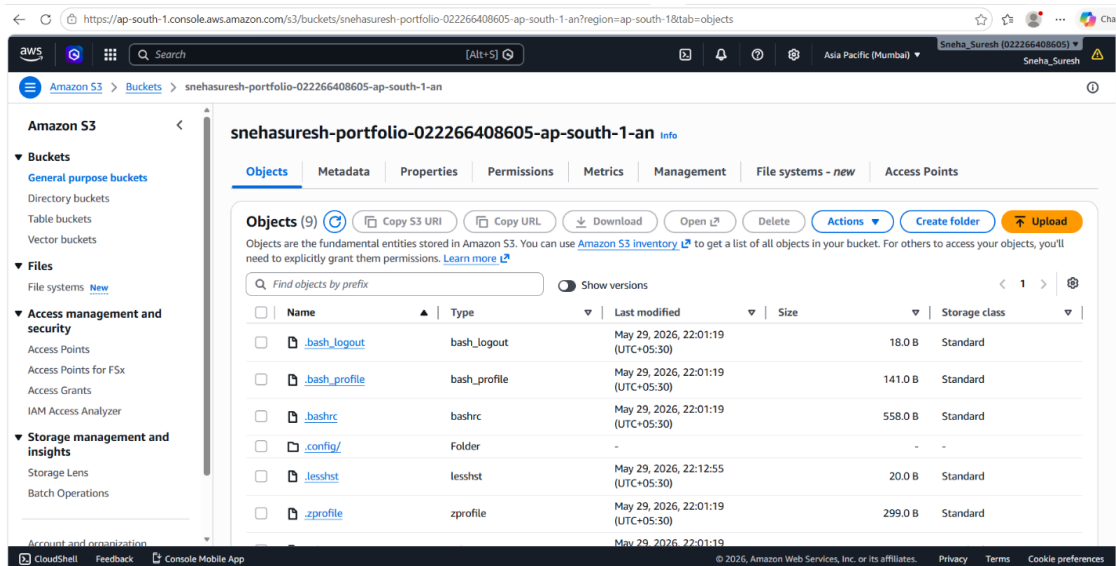


Figure 2.1: S3 bucket snehasuresh-portfolio-022266408605-ap-south-1-an — 9 objects uploaded successfully

Live Site via S3 Endpoint (HTTP)

Portfolio successfully rendered on the S3 static website endpoint before CloudFront setup, confirming correct uploads and bucket policy.

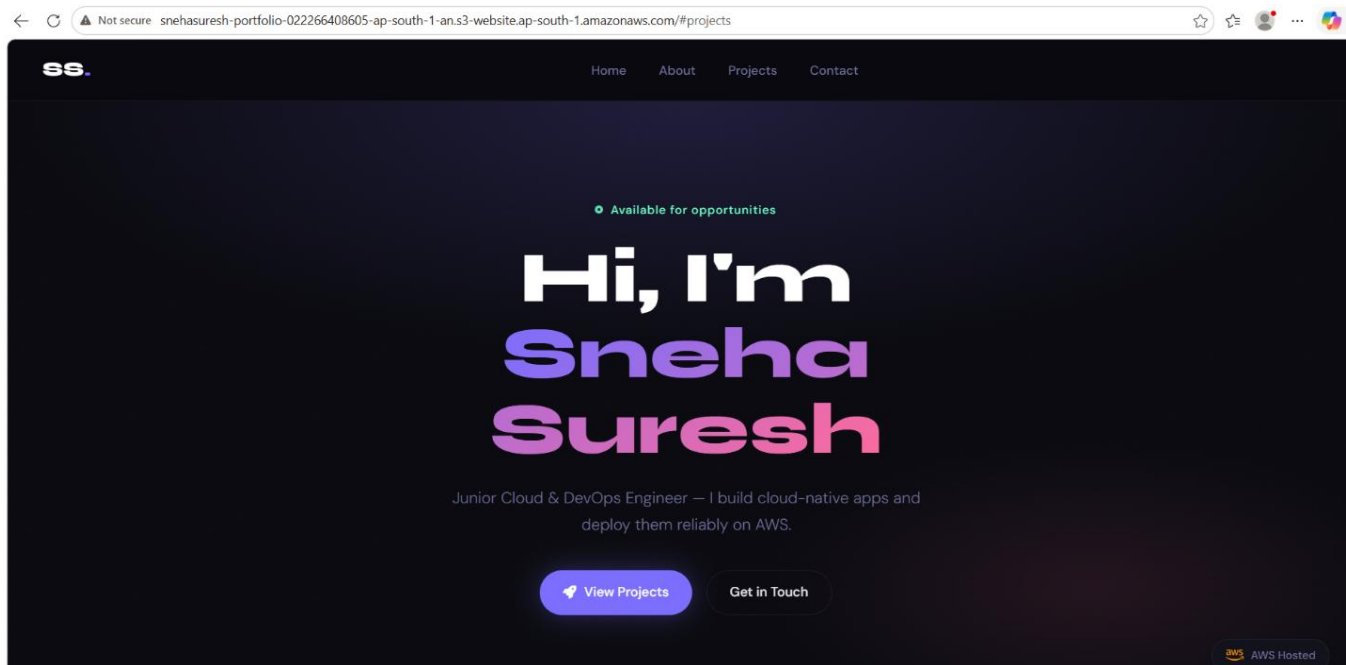


Figure 2.2: Portfolio on S3 website endpoint — <http://snehasuresh-portfolio-022266408605-ap-south-1-an.s3-website.ap-south-1.amazonaws.com/#/projects>

Key Learnings

- Configured S3 bucket for static website hosting with index.html as default document.
- Applied IAM bucket policies to allow public read access (s3:GetObject) for all objects.
- Used AWS CLI sync command to upload local project files to S3 efficiently.
- Understood the S3 website endpoint structure and difference from the S3 REST endpoint.
- Practiced disabling Block Public Access settings to make a bucket publicly accessible.

Mini Project — Deploy a Static Portfolio Website on AWS

Project Overview

A responsive single-page portfolio was built with HTML5 & CSS3, hosted on AWS S3, and served securely via Amazon CloudFront CDN with HTTPS. This mini project ties together all Week 2 skills: S3 hosting, bucket policies, CloudFront CDN, and AWS CLI.

What Was Built

- Fully responsive dark-themed single-page portfolio — Hero, About, 3 Project Cards, Contact footer.
- Google Fonts (Syne + DM Sans) + FontAwesome 6 icons used throughout.
- Hosted on AWS S3 (ap-south-1) with public bucket policy.
- Served via CloudFront CDN with Redirect HTTP to HTTPS enabled.

Technologies Used

- HTML5 & CSS3 — responsive dark-themed portfolio design
- Google Fonts (Syne + DM Sans) — professional typography
- FontAwesome 6 — icon library
- AWS S3 (ap-south-1) — static website hosting
- AWS CloudFront (ID: E32GCGNK01LRYW) — CDN + HTTPS redirect
- IAM Bucket Policy — s3:GetObject for all objects
- Git & GitHub — version control
- AWS CLI — `aws s3 sync`

Deployment Steps

Step 1 — Local Development

Built index.html, styles.css, and assets/ locally and tested in browser.

Step 2 — Push to GitHub

```
git init && git add . && git commit -m "initial portfolio"
git remote add origin https://github.com/sneh20047078/My_Portfolio.git
git push -u origin main
```

Step 3 — S3 Bucket Setup & Upload

- Created S3 bucket, disabled Block All Public Access, enabled Static Website Hosting.
- Applied s3:GetObject bucket policy for public read.

```
aws s3 sync . s3://snehasuresh-portfolio-022266408605-ap-south-1-an --acl public-read
```

Step 4 — CloudFront Distribution

- CloudFront → Create distribution → Origin: S3 website endpoint.
- Viewer Protocol: Redirect HTTP to HTTPS. Default root: index.html.
- Distribution ID: E32GCGNK01LRYW — Domain: d30ll7dxewuby5.cloudfront.net

Live URLs

CloudFront HTTPS: <https://d30ll7dxewuby5.cloudfront.net>

S3 Endpoint: <http://snehasuresh-portfolio-022266408605-ap-south-1-an.s3-website.ap-south-1.amazonaws.com>

GitHub: https://github.com/sneh20047078/My_Portfolio

Screenshots — Mini Project

Figure 1 — Live Site via CloudFront (HTTPS)

Portfolio is live at <https://d30ll7dxewuby5.cloudfront.net>. Browser shows HTTPS padlock — CloudFront is securing the site.

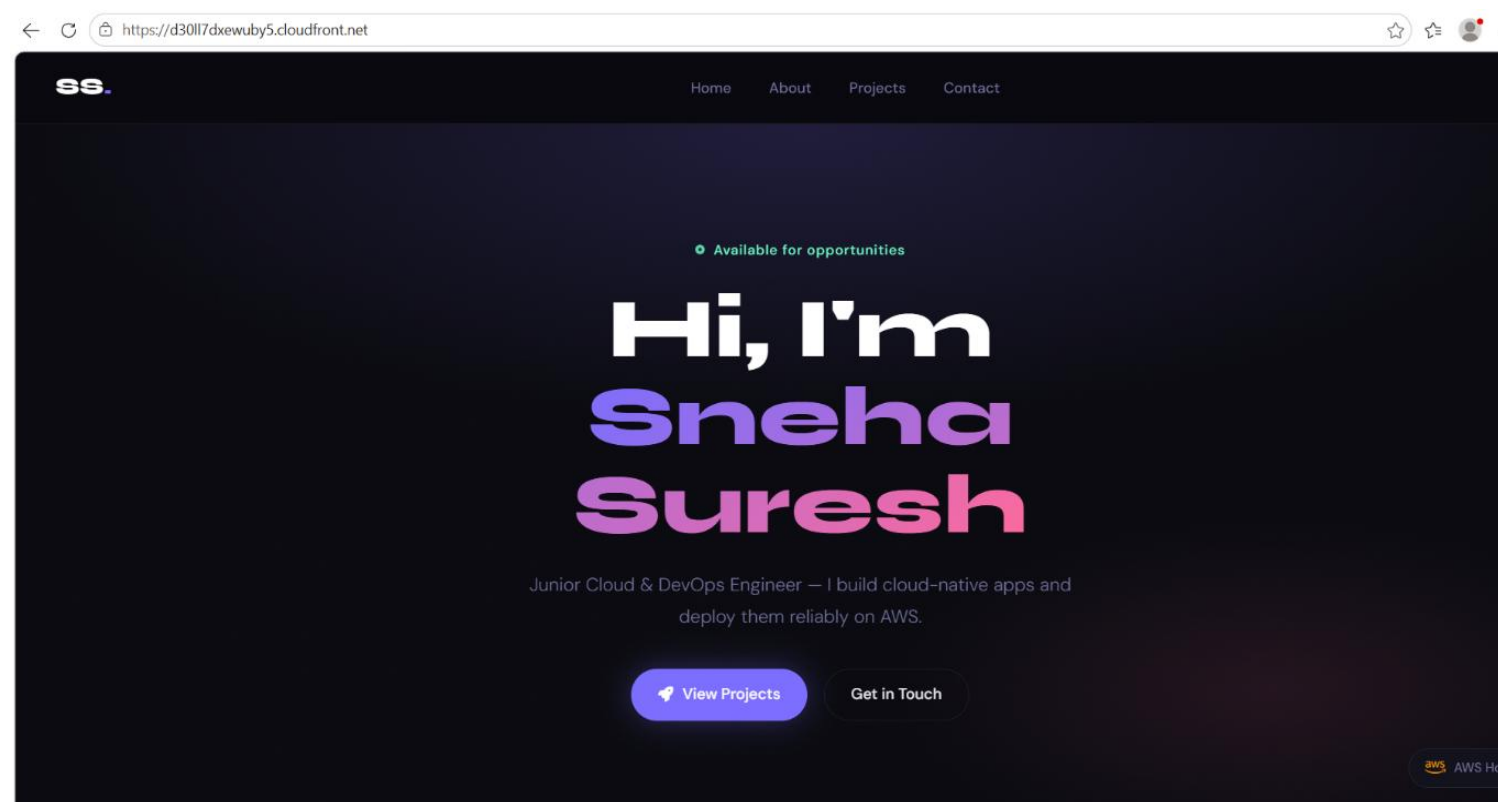


Figure 3.1: Portfolio live via CloudFront HTTPS — <https://d30ll7dxewuby5.cloudfront.net>

Figure 2 — S3 Bucket Objects

All portfolio files successfully uploaded to the S3 bucket in Mumbai region (ap-south-1).

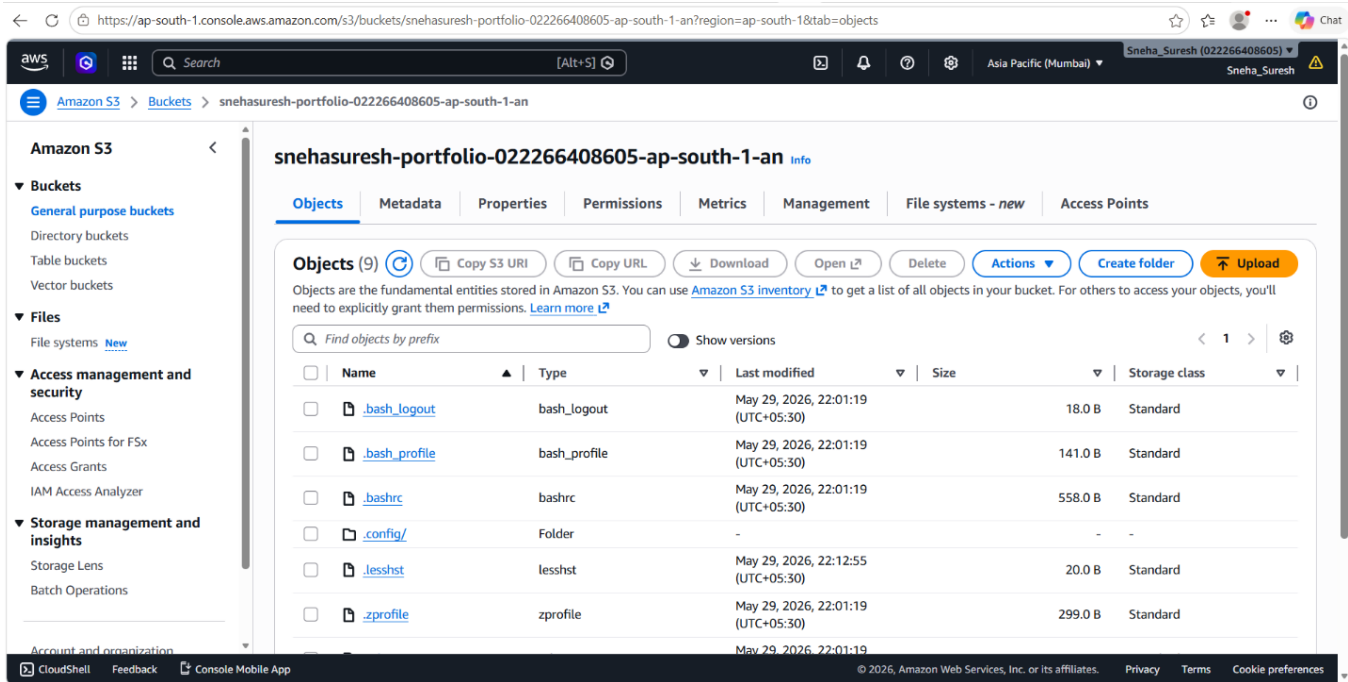


Figure 3.2: S3 bucket with 9 portfolio objects — snehasuresh-portfolio-022266408605-ap-south-1-an

Figure 3 — CloudFront Origins Tab

CloudFront distribution Myportfolio has the S3 bucket configured as its origin (Type: S3). Distribution domain: d30ll7dxewuby5.cloudfront.net.

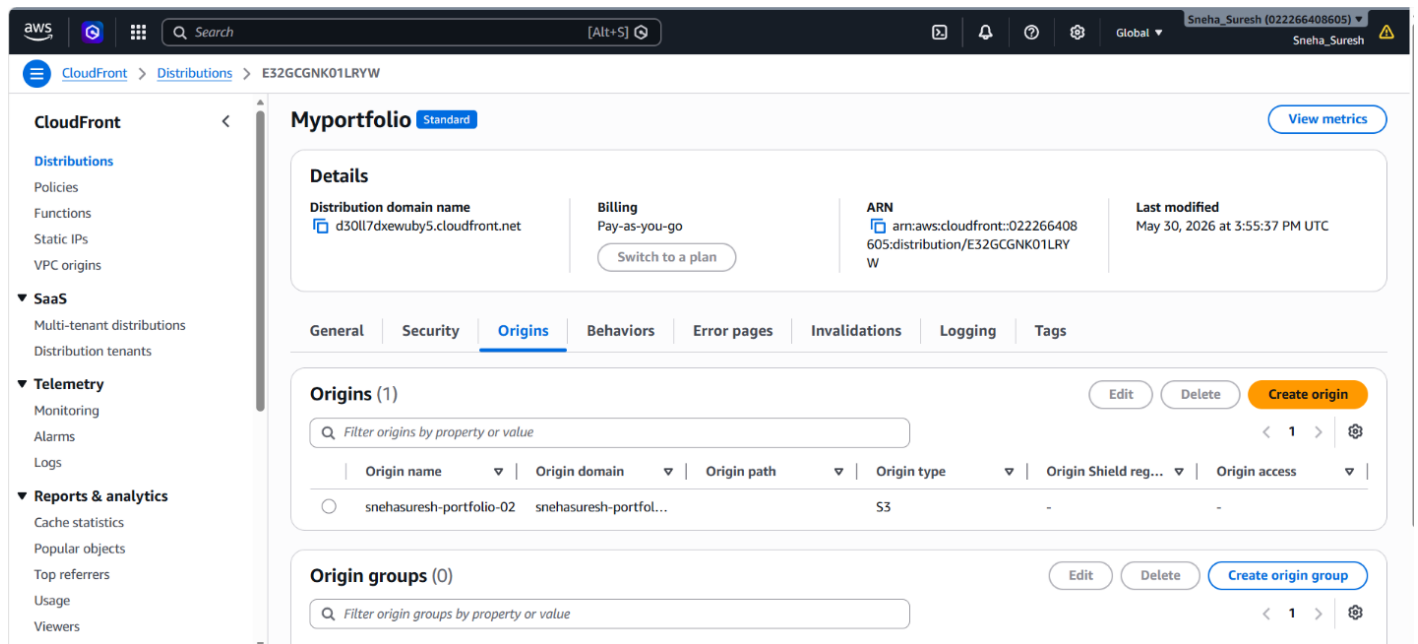


Figure 3.3: CloudFront Origins — S3 bucket snehasuresh-portfolio-02 as origin, ID: E32GCGNK01LRYW

Figure 4 — CloudFront General Settings

General tab confirms default root object index.html, HTTP/2 enabled, Price class set to all edge locations for best performance globally.

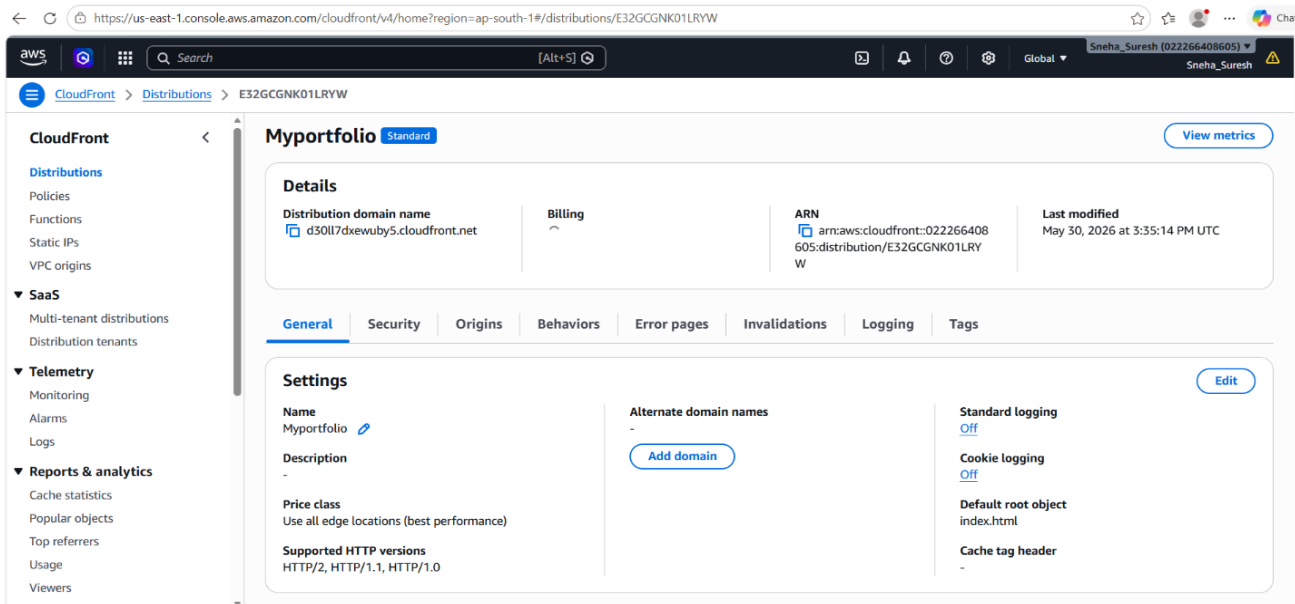


Figure 3.4: CloudFront General Settings — domain d30ll7dxewuby5.cloudfront.net, default root: index.html

Figure 5 — CloudFront Distribution Status: Enabled

Distributions list shows distribution E32GCGNK01LRYW with Status: Enabled — the site is fully deployed and live.

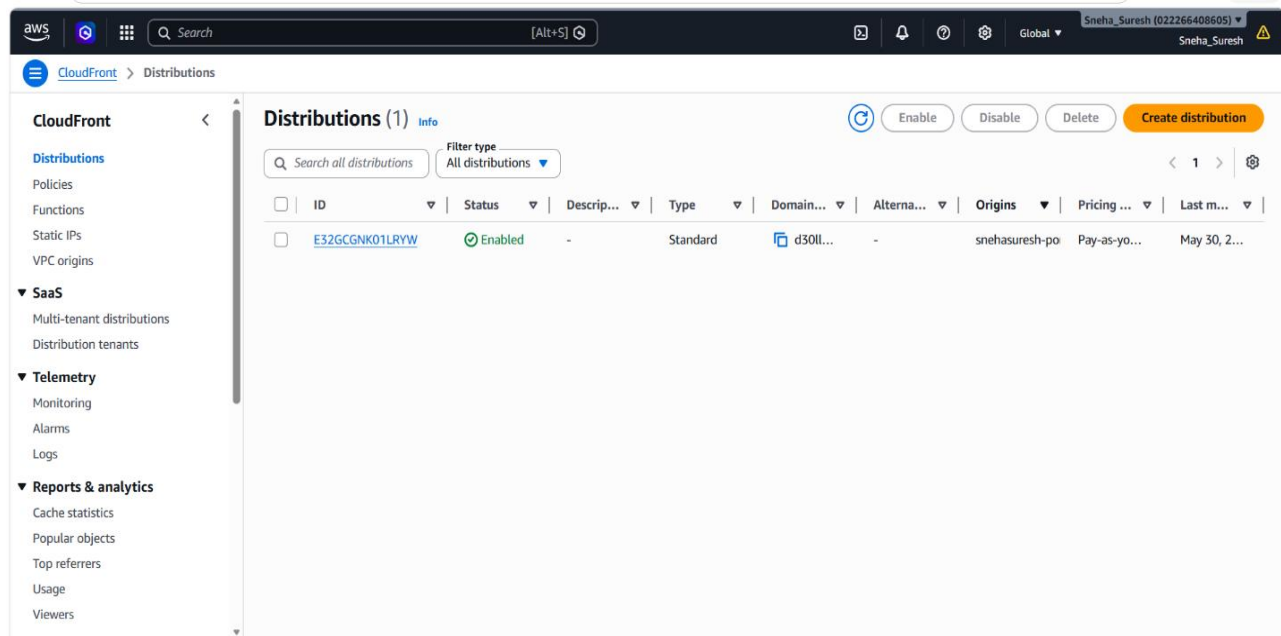


Figure 3.5: CloudFront Distributions list — E32GCGNK01LRYW Status: Enabled, domain d30ll7dxewuby5.cloudfront.net

Key Learnings

- End-to-end experience deploying a static portfolio using S3 + CloudFront on AWS.
- Understood how CloudFront caches content at global edge locations for faster delivery.
- Configured HTTPS by enabling Redirect HTTP to HTTPS in CloudFront viewer protocol policy.
- Applied IAM bucket policies for secure controlled public access to S3 objects.
- Used Git and GitHub for version control and as a public code deliverable.